



LeetCode 47: Permutations II

1. Problem Title & Link

Title: LeetCode 47: Permutations II

Link: <https://leetcode.com/problems/permutations-ii/>

2. Problem Statement (Short Summary)

Given an integer array:

nums

which may contain duplicate values, return all possible **unique permutations**.

The result can be returned in any order.

3. Examples (Input → Output)

Example 1

Input:

nums = [1,1,2]

Output:

```
[  
  [1,1,2],  
  [1,2,1],  
  [2,1,1]  
]
```

Example 2

Input:

nums = [1,2,3]

Output:

```
[  
  [1,2,3],  
  [1,3,2],  
  [2,1,3],  
  [2,3,1],  
  [3,1,2],  
  [3,2,1]  
]
```

4. Constraints

- $1 \leq \text{nums.length} \leq 8$



- $-10 \leq \text{nums}[i] \leq 10$

5. Core Concept (Pattern / Topic)

Backtracking + Duplicate Skipping

Related Topics:

- Recursion
- DFS
- Permutations
- Backtracking

6. Thought Process (Step-by-Step Explanation)

First Idea

Generate all permutations.

Example:

[1,1,2]

Normal permutation generation gives:

[1,1,2]

[1,2,1]

[1,1,2]

[1,2,1]

[2,1,1]

[2,1,1]

Duplicates appear.

Brute Force

Generate all permutations.

Store them in:

Set

to remove duplicates.

Complexity:

$O(n! \times n)$

Extra memory needed.

**Better Idea**

Avoid generating duplicates in the first place.

Key Observation

Sort the array first.

Example:

nums = [1,1,2]

After sorting:

[1,1,2]

Now duplicate elements become adjacent.

Duplicate Skipping Rule

Suppose:

[1,1,2]

At the same recursion level:

If first:

1

has not been used,

then we should not choose the second:

1

first.

Rule:

if $i > 0$

and $\text{nums}[i] == \text{nums}[i-1]$

and not used[i-1]

skip current element.

Why Does This Work?

Consider:

[1a,1b,2]

Choosing:

1a

first and then:

1b

already generates every valid permutation.

Starting with:



1b

would create duplicate results.

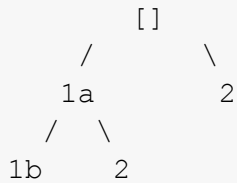
So we skip it.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

1a 1b 2

Tree:



Skip:

1b as first choice

because:

1a already covers it

8. Pseudocode

```

sort nums

used = [False]*n

backtrack():

    if permutation complete:
        save answer

    for each index:

        if already used:
            continue

        if duplicate condition:
            continue

        choose element

        recurse

        undo choice

```

9. Code Implementation

✓ Python



```
class Solution:

    def permuteUnique(self, nums):

        nums.sort()

        result = []

        used = [False] * len(nums)

        def backtrack(path):

            if len(path) == len(nums):

                result.append(path[:])
                return

            for i in range(len(nums)):

                if used[i]:
                    continue

                if (
                    i > 0 and
                    nums[i] == nums[i - 1] and
                    not used[i - 1]
                ):
                    continue

                used[i] = True
                path.append(nums[i])

                backtrack(path)

                path.pop()
                used[i] = False

            backtrack([])

        return result
```

✓ Java

```
import java.util.*;

class Solution {

    public List<List<Integer>>
    permuteUnique(int[] nums) {
```



```
Arrays.sort(nums);

List<List<Integer>> result =
    new ArrayList<>();

boolean[] used =
    new boolean[nums.length];

backtrack(
    nums,
    used,
    new ArrayList<>(),
    result
);

return result;
}

private void backtrack(
    int[] nums,
    boolean[] used,
    List<Integer> path,
    List<List<Integer>> result) {

    if (path.size() == nums.length) {

        result.add(
            new ArrayList<>(path)
        );

        return;
    }

    for (int i = 0;
        i < nums.length;
        i++) {

        if (used[i])
            continue;

        if (
            i > 0 &&
            nums[i] == nums[i - 1] &&
            !used[i - 1]
        )
            continue;

        used[i] = true;

        path.add(nums[i]);
```



```
        backtrack(
            nums,
            used,
            path,
            result
        );

        path.remove(
            path.size() - 1
        );

        used[i] = false;
    }
}
```

10. Time & Space Complexity

Time Complexity

Worst Case:

$O(n \times n!)$

Generating all unique permutations.

Space Complexity

$O(n)$

Recursion stack + used array.

(Result storage not counted.)

11. Common Mistakes / Edge Cases

Mistake 1

Not sorting first.

Duplicate skipping won't work.

Mistake 2

Using:

`nums[i] == nums[i-1]`

alone.

Need:

and not `used[i-1]`

Mistake 3



Forgetting backtracking cleanup.

```
path.pop()
```

```
used[i] = False
```

12. Detailed Dry Run

```
Input:
nums = [1,1,2]
Sorted:
[1,1,2]

Level 1
Choose:
1(index 0)
Path:
[1]

Level 2
Choose:
1(index 1)
Path:
[1,1]

Level 3
Choose:
2
Permutation:
[1,1,2]

Backtrack
Choose:
2
Path:
[1,2]
Then:
1
Permutation:
[1,2,1]

Root Level
Skip:
1(index 1)
because:
nums[1] == nums[0]

and

used[0] == False
```




```
Duplicate avoided.
```

```
Choose:
```

```
2
```

```
Then:
```

```
1
```

```
1
```

```
Permutation:
```

```
[2,1,1]
```

```
Final:
```

```
[
```

```
    [1,1,2],
```

```
    [1,2,1],
```

```
    [2,1,1]
```

```
]
```

13. Common Use Cases

Seating Arrangements

Unique arrangements with repeated people/groups.

Password Generation

Generate distinct combinations.

Scheduling Problems

Unique task orders with repeated tasks.

Game State Generation

Unique move sequences.

14. Common Traps

- Not sorting input
- Wrong duplicate check
- Forgetting to backtrack
- Using Set unnecessarily

15. Builds To (Related Problems)

1. LC 46 — Permutations
2. LC 47 — Permutations II
3. LC 77 — Combinations
4. LC 90 — Subsets II



5. LC 40 — Combination Sum II

16. Alternate Approaches

Approach	Time	Space	Notes
Generate All + Set	$O(n \times n!)$	Large	Inefficient
Backtracking + Skip Duplicates	$O(n \times n!)$	$O(n)$	Optimal

17. Why This Solution Works

Sorting places duplicate values together. By skipping a duplicate when its previous copy has not been used in the current recursive path, we ensure that each unique permutation is generated exactly once.

18. Variations / Follow-Up Questions

Follow-Up 1

Generate all permutations.

→ LC 46

Follow-Up 2

Generate unique subsets.

→ LC 90

Follow-Up 3

Generate unique combinations with target sum.

→ LC 40